

Autonomous Evaluation of Persistent-Contract Agent Workflows: A Conditional AutoMLR Draft

Anonymous Authors

August 30, 2026

Eligibility status. This draft is intentionally not submission-ready. The qualifying ML result required by AutoMLR does not yet exist: the five-task, 14-run study described below is pre-registered but unexecuted. The abstract-registration status must also be verified against the live portal. Part II is expected by the workshop to be human-written; AI-generated material below is marked as factual drafting notes requiring human rewrite.

Abstract

Conditional abstract – do not submit as written. We study whether a persistent-contract hierarchy for language-agent execution preserves requirements and reduces repair scope relative to a competent flat ChatGPT+Claude-Code workflow. The autonomous system under study maintains a human-approved Plan, multi-view Phase contracts, just-in-time execution stacks, versioned evidence, and verification outcomes that select local repair, replan, or escalation. We pre-register five Harbor tasks spanning iterative specification refinement, live stateful systems, and scientific certification, with ten primary paired runs and four independent-account replications. **[QUALIFYING RESULT REQUIRED HERE: replace only with evidence generated from the frozen run store.]** This paper can qualify for AutoMLR only if the autonomous system makes a decisive contribution to that primary ML result and the intervention log supports that claim.

Part I — Autonomous ML research result (maximum 4 pages)

Status: protocol complete; qualifying result pending.

1. Research question

Long-horizon language-agent systems increasingly use plans, subagents, memory, and verification, but it remains unclear whether a persistent hierarchical control layer improves task execution enough to justify its overhead. We test a mechanism-level hypothesis: compared with a competent flat workflow using the same ChatGPT Main and Claude Code executor, does AI-Native Workflow better preserve accumulated requirements and localize recovery on tasks whose specifications or live state evolve over time?

The method is itself an agent-management system rather than a new base model. Its canonical hierarchy is

Goal → Plan → Phase → DIC → EPS → Prompt/Run → Action.

A Phase has one durable multi-view DIC. `REQUESTS.md` stores concrete goals, while `PLAN.md` stores prompt dependencies, parallel subagents, merge points, and conditional repair branches. EPS instances materialize that plan against current state. Verification returns accept, repair, replan, or escalate.

2. Frozen paired study

The evaluation is deliberately small because it is a workshop proof of concept executed with subscription resources rather than an API-scale sweep. ChatGPT Plus is Main in every run. Claude Code authenticated through Claude Pro A is the primary executor; Claude Pro B is reserved for the pre-declared hard-task replications.

We compare two conditions. **F (Flat)** uses the same ChatGPT+Claude stack and permits ordinary planning, iterative debugging, and local tests, but omits the package-resident DIC/EPS hierarchy, persistent Request ledger, dependency-local invalidation, and structured recovery state. **W (Workflow)** uses the full AI-Native system. F is therefore a strong non-hierarchical baseline rather than a one-shot prompt.

Table 1: Frozen candidate tasks and planned repetitions. No score is a result yet.

Task	Role	Scored runs
env_manager	5-checkpoint easier evolving-spec control	F/W Pro A
database_migration	dependencies, rollback, accumulated requirements	F/W A + F/W B
recli	8-checkpoint hard architectural evolution	F/W A + F/W B
live-database-cutover	live state, consistency, cutover verification	F/W A
certified-sparse-regression	scientific optimization/certification transfer	F/W A

T1–T3 are SlopCodeBench tasks designed around iterative specification refinement and regression measurement [Orlanski et al., 2026]; T4 is a Terminal-Bench task [Merrill et al., 2026]; T5 is from Terminal-Bench-Science [Terminal-Bench Science Team, 2026]. The total planned scored workload is 14 runs. Exact task versions are locked before execution; oracle solutions are isolated to environment qualification and unavailable to scored agents.

3. Outcomes and evidence discipline

The primary evidence consists of official benchmark reward/pass, SlopCodeBench strict pass rate by checkpoint, regression/erosion, rework, repair scope, Main turn count, executor prompt/run count, and wall time. Token counts are included only when the subscription surfaces expose them reliably. With five task families we report all runs and paired/checkpoint traces; we do not turn the study into a population-level significance claim.

All scored evidence must enter a machine-readable run store. Manuscript tables are generated from raw Harbor/verifier records; no aggregate number is hand-copied from a chat summary. Failed attempts and negative results are retained. This is particularly important for AutoMLR because eligibility depends on a real research result, not on a narrative that an autonomous system “did research.”

4. Qualifying-result insertion contract

[NO RESULT EXISTS YET.]

After Phases 03–05, this subsection must be replaced by a concise result containing:

1. the exact per-task F/W outcomes and the two Pro-B replications;
2. checkpoint trajectories for the three SlopCodeBench tasks;
3. a bounded claim describing what improved, stayed neutral, or regressed;
4. resource differences and failures;
5. an intervention audit establishing whether the autonomous system conducted the decisive research steps required by AutoMLR.

If the study is null, inconsistent, or mainly human-driven, the correct action is to mark the paper ineligible rather than manufacture a qualifying conclusion.

5. Formal support and limitations

Under complete declared dependencies and version-bound evidence, a repair that changes keys Δ need invalidate only the dynamic impact cone of nodes that read those keys and their descendants; the cone is worst-case tight given footprint information alone. If a persisting observable fault is missed at each gate with conditional probability at most α , survival through k gates is at most α^k . Conversely, if the evidence observation aliases a successful and unsuccessful state, no evidence-only verifier can attain both zero false acceptance and zero false rejection. These statements motivate the mechanism but do not substitute for empirical evidence.

Part II — Autonomous system/design (maximum 4 pages)

HUMAN AUTHORSHIP / REWRITE REQUIRED BEFORE SUBMISSION

The workshop states that Part II is expected to be human-written. The paragraphs below are therefore **verified factual notes and a drafting structure**, not venue-compliant final prose.

1. System boundary and persistent authority

[HUMAN REWRITE REQUIRED] The system stores canonical state in `.ai-workflow/STATE.json`. A human approves a multi-phase Plan. Main owns integration, architecture and publication decisions; Theory owns formal/novelty work; Experiment owns benchmark methodology and evidence. Ordinary machine execution occurs below the Plan boundary. Material objective, scientific-claim, protocol, destructive/external, credentials/private-system, paid-compute, release and submission changes are explicit escalation categories.

2. DIC, execution plan and EPS

[HUMAN REWRITE REQUIRED] Each Phase has one multi-view DIC. README gives the overall summary; REQUESTS records specific goals and acceptance targets; PLAN records actual prompt/run nodes, dependencies, parallel subagent batches, merge/review points and conditional branches. ARCHITECTURE and TEST_MATRIX are optional views, and users can register custom views. The EPS is not another high-level plan: it instantiates durable Plan nodes just in time with current context, model surface, attempt, outputs and stop conditions.

3. Verification and recovery

[HUMAN REWRITE REQUIRED] Verification produces accept, repair, replan or escalate. Local deterministic checks occur automatically; fresh independent review is reserved for material claims/release boundaries. A repair normally preserves the DIC and supersedes only affected EPS/run instances. Phase reports are mandatory, flat, capped at ten files by default, and include key merged raw evidence as well as human synthesis.

4. Research-loop autonomy for the qualifying result

[HUMAN REWRITE REQUIRED AFTER RUNS] The final paper must state, chronologically and specifically, which of the following were autonomous for Part I: hypothesis formation, benchmark selection, protocol design, environment implementation, run dispatch, debugging, analysis, claim selection, figure/table generation, and manuscript drafting. The current package records a machine-authored protocol and code-agent Phase 01 plan, but the qualifying experiment has not yet occurred; no stronger autonomy claim is justified now.

5. Human intervention accounting

[HUMAN REWRITE REQUIRED] Human interventions must be derived from the package event/history and actual experiment logs. Examples that must be disclosed if they occur include task replacement, model/account substitution, benchmark workaround, changed stopping rules, reruns after observing results, or manual interpretation that materially determines the conclusion. Human authors remain responsible for all claims, citations, and submission choices.

6. Safety and resource controls

[HUMAN REWRITE REQUIRED] The current protocol forbids paid Modal/Daytona/API compute without separate authorization and keeps credentials outside Git. Oracle solutions are isolated from scored workspaces. GitHub publication and workshop submission are distinct external boundaries. The workflow is an audit/control layer, not a security sandbox.

Part III — Reflections (maximum 1 page)

HUMAN AUTHORSHIP / REWRITE REQUIRED BEFORE SUBMISSION

A compliant final Part III should be written after the qualifying study and should address at least four questions: (1) which scientific decisions the autonomous system made that the human authors would otherwise have made; (2) where the system optimized the wrong proxy or required intervention; (3) whether persistent contracts and raw evidence made the result easier to audit or merely increased bureaucracy; and (4) what should remain a human judgment even if the agent can execute most research steps.

One provisional lesson can be stated factually from the current package: externalizing requests, execution graphs, evidence and repair history makes it easier to identify *where* human intervention occurs. That is useful for disclosure, but it does not itself establish autonomous research quality. AutoMLR eligibility should therefore be treated as an empirical property of the completed research trajectory, not a label attached to an agent harness.

References

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.

Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Aws Albarghouthi, and Frederic Sala. Slopcodabench: Measuring code erosion under iterative specification refinement. *arXiv preprint arXiv:2603.24755*, 2026. URL <https://arxiv.org/abs/2603.24755>.

Terminal-Bench Science Team. Terminal-bench science: Evaluating ai agents on computational workflows in the natural sciences. <https://github.com/harbor-framework/terminal-bench-science>, 2026.